Document number: P0960R0
Audience: EWG

Ville Voutilainen
2018-02-12

Allow initializing aggregates from a parenthesized list of values

Abstract

This paper proposes allowing initializing aggregates from a parenthesized list of values; that is, Aggr(val1, val2) would mean the same thing as Aggr{val1, val2}. This is a language fix for the problem illustrated in N4462.

"This is a completely new way of initializing aggregates and adds complexity to the language"

I couldn't disagree more. First of all, this is not a new way of initializing aggregates, we can already initialize aggregates with parentheses:

```
struct X {int a, b;};
int main()
{
    X x{};
    X x2(x); // see, an aggregate initialized with parentheses
}
```

Second, this makes the language more regular. I can use X{1, 2} to initialize an aggregate temporary, but X(1, 2) just doesn't work. The only reason for not making it work is wishful thinking that everybody would use braced-initialization everywhere, but practical deployment experience of C++11 shows that that's indeed wishful thinking. The library facilities mentioned in N4462 don't use braced-initialization, and can't be made to use it because that would be a breaking change. All the lack of support for parenthesis-initialization for aggregates leads to is programmers adding constructors where they shouldn't need to add them, or performing an additional copy/move because that's a way to work around the problem.

Nested aggregates

Some have asked whether nested aggregates should support initialization from a parenthesized list of values that contains nested parenthesized lists of values. Akin to

```
struct X {int a, b;};
struct Y {X x;};
void f() { Y y{{1,2}};}
```

the question seems to be whether it should be possible to write Y y((1,2)) and have it mean the same as Y y{{1,2}}. The answer is no. The nested parentheses already have a meaning, and there is no use case for adding new meaning to nested parentheses.

"Can of worms"

Johannes Schaub provided the following example:

```
struct A { int x; };
A a;
A b(a);
```

The question was 'Will this try to initialize ".x" by "a", or will it copy a to b?'. The answer is obvious, since it will copy a to b, and it already does so today. This proposal doesn't change that in any way, and the answer is consistent with "X(a) means X{a} for aggregates".

Johannes had another example:

```
struct A { int x; };

struct B {
   operator A();
};

B b;
A a(b);
```

The question was 'Does it try to convert the "b" to "int" (initializing .x) or to "A"?' The answer is that it will do exactly what A a{b}; would do. There is no can of worms here.

"This introcudes new vexing parses"

I don't see how. Every vexing parse I can come up with is already vexing today; struct X{int a, b;}; int main() { X x(int(), int());} is a valid C++ program where main contains a declaration of a function.

Designated initializers

Should X(.a=42, .b=666) be valid? I don't see a pressing need to make it so; designated initializers are a C-compatibility feature, and that syntax is not a thing in C. For the general "X(a) means X{a}", it's plausible to support designated initializers.

"Please, please make it turn off narrowing prevention!"

There was a strong suggestion on std-proposals to make the parenthesized syntax turn off narrowing prevention. EWG feedback on that point is welcomed.

Arrays

Should arrays be constructible with parentheses? For the sake of "X(a) means X{a}", yes. If there is some reason why arrays should be an exception, the author of this paper won't lose sleep over it.

Let's go back to that additional copy/move

An oft-suggested work-around for make_shared<Aggr>(1,2); not working is make_shared<Aggr>(Aggr{1,2});. This will incur an extra move. That move might not be cheap. That move might not be valid. A part of our emplace functions' rationale is to be able to do in-place construction to construct even non-copyable and non-movable types. With aggregates, this is currently not possible.

"I still don't understand why we should do this"

This proposal allows make_shared, make_unique, emplace functions, allocator::construct, and a whole host of other things to Just Work with aggregate types. We need no library hacks to try and use braced initialization if parenthesized initialization isn't valid, the library facilities will sprout proper aggregate support as soon as this language change is made.